

云从科技大前端工程师（AI交互方向）

一面（技术基础） · 1-6

1. 请简述一下 SSE（Server-Sent Events）和 WebSocket 的区别，为什么 AI 聊天应用通常选择 SSE？
2. 在处理 AI 返回的 Markdown 字符串时，如何安全地将其渲染到页面上？
3. 请手写一个函数，用于解析 SSE 返回的 data 数据流并提取出 content 字段。
4. 谈谈 CSS 中实现“打字机”光标闪烁效果的几种方式。
5. 如何利用 Fetch API 的 ReadableStream 实现流式读取响应体？
6. 介绍一下 Promise.all 和 Promise.allSettled 的区别，以及在 AI 多模型并行调用场景下的选型。

二面（深入原理与项目） · 1-8

1. AI 聊天界面的长列表通常会遇到性能瓶颈，你如何实现一个支持流式输出的虚拟列表？
2. 在前端如何实现 Prompt 的版本管理和动态替换变量？
3. 谈谈你对 Function Calling（函数调用）前端实现流程的理解。
4. 如何优化大模型首字响应时间（TTFT）的前端感知？
5. 如果 AI 返回的内容包含复杂的数学公式或 Mermaid 图表，前端如何集成相关插件？
6. 你们是如何处理 AI 会中的上下文长度限制（Context Window）的？
7. 实现一个带并发限制的 Promise 调度器，模拟同时调用多个 AI 代理。
8. 谈谈前端如何防范 Prompt 注入攻击？

三面（架构与综合能力） · 1-6

1. 如果让你设计一个企业级的 AI Agent 工作流编排系统（类似 LangFlow），你会如何设计前端架构？
2. 在 AI 交互中，如何定义一套衡量“前端交互质量”的指标体系？
3. 如何解决 AI 场景下前端资源体积过大（如集成 WebLLM 或大型渲染库）的问题？
4. 谈谈 AI 时代下，前端工程师的角色发生了哪些转变？你如何看待“AI 会取代前端”这个观点？
5. 聊聊你在过去一年中遇到的最难的 AI 相关前端技术挑战，你是如何解决的？
6. 如果大模型支持 1M 的上下文，前端在数据展示和交互上需要做哪些适配？

一面（技术基础）

1. 请简述一下 SSE (Server-Sent Events) 和 WebSocket 的区别，为什么 AI 聊天应用通常选择 SSE?

难度：★★

考点：网络协议、流式传输

答案要点：

SSE 是一种基于 HTTP 的单向推送技术，而 WebSocket 是基于 TCP 的全双工通信协议。AI 场景选择 SSE 主要是因为其轻量且原生支持流式输出。

- 协议层：SSE 基于标准 HTTP 协议，兼容性好，支持断线重连；WebSocket 是独立协议，需握手升级。
- 方向性：SSE 仅支持服务端向客户端推送；WebSocket 支持双向实时通信。
- 场景匹配：LLM 生成回复是单向流式的，SSE 的 text/event-stream 格式天然契合。
- 资源消耗：SSE 更加轻量，在仅需服务端推送的场景下，服务器开销更小。

常见坑：

- 误认为 SSE 支持二进制传输（其实只支持文本）。
- 忽略了浏览器对 HTTP/1.1 下 SSE 最大连接数（6个）的限制。

追问：

- 如果需要客户端在传输过程中中断请求，前端应该如何操作？

2. 在处理 AI 返回的 Markdown 字符串时，如何安全地将其渲染到页面上？

难度：★★

考点：前端安全、XSS 攻击、DOM 渲染

答案要点：

渲染 AI 生成的内容必须经过“解析-过滤-渲染”三个阶段，核心是防止恶意脚本注入。

- 解析器选择：使用 marked 或 markdown-it 将字符串转为 HTML。
- 消毒处理：必须使用 DOMPurify 等库对生成的 HTML 进行清洗，剔除 script、onerror 等危险标签和属性。
- 代码高亮：结合 Prism.js 或 highlight.js 处理代码块。
- 性能优化：对于长文本，考虑增量解析或分片渲染。

常见坑：

- 直接使用 dangerouslySetInnerHTML 而不进行任何过滤。
- 忽略了 AI 可能被诱导生成带有恶意链接的 Markdown。

追问：

- 如何实现类似 ChatGPT 的代码块复制功能？

3. 请手写一个函数，用于解析 SSE 返回的 data 数据流并提取出 content 字段。

难度：★★

考点：字符串处理、流式数据解析

答案要点：

SSE 返回的数据通常以 `data:` 开头，以 `\n\n` 结尾，需要处理多行拼接和 JSON 解析。

- 匹配前缀：识别 `data:` 标识。
- 异常处理：处理 `[DONE]` 结束标志或非 JSON 格式的情况。
- 累加逻辑：在流式传输中，可能一个数据包包含多个 data 块。

代码块

```
1  function parseSSEChunk(chunk) {
2    const lines = chunk.split('\n');
3    const results = [];
4    for (const line of lines) {
5      if (line.startsWith('data: ')) {
6        const data = line.slice(6);
7        if (data === '[DONE]') break;
8        try {
9          const parsed = JSON.parse(data);
10         results.push(parsed.choices[0]?.delta?.content || '');
11       } catch (e) {
12         console.error('Parse error', e);
13       }
14     }
15   }
16   return results.join('');
17 }
```

常见坑：

- 忽略了一个 chunk 中可能包含多条 data 数据的情况。
- 没处理数据包被截断导致 JSON 解析失败的情况。

4. 谈谈 CSS 中实现“打字机”光标闪烁效果的几种方式。

难度：★

考点：CSS 动画、伪元素

答案要点：

可以通过 CSS 动画控制伪元素的 border 或 opacity 属性来实现。

- 伪元素法：使用 `::after` 模拟光标，设置 border-right。

- 关键帧动画：定义 @keyframes 改变透明度（0% 到 100%）。
- 步进函数：使用 steps() 函数让动画具有跳动感，而非平滑过渡。

常见坑：

- 动画导致页面频繁重排（建议用 opacity 触发合成层）。

追问：

- 当文本换行时，如何保证光标始终在最后一个字符后面？

5. 如何利用 Fetch API 的 ReadableStream 实现流式读取响应体？

难度：★★★

考点：Fetch API、Streams API

答案要点：

Fetch 的 response.body 是一个 ReadableStream，可以通过 getReader() 逐块读取数据。

- 获取读取器：const reader = response.body.getReader()。
- 循环读取：使用 while(true) 配合 reader.read()。
- 解码数据：使用 TextDecoder 将 Uint8Array 转换为字符串。
- 状态控制：通过 done 变量判断流是否结束。

常见坑：

- 忘记处理网络异常导致的流中断。
- 没使用 TextDecoder 导致中文字符被截断乱码。

6. 介绍一下 Promise.all 和 Promise.allSettled 的区别，以及在 AI 多模型并行调用场景下的选型。

难度：★★

考点：异步编程、并发控制

答案要点：

Promise.all 具有原子性，一个失败则全部失败；allSettled 会等待所有结果返回，无论成功失败。

- 状态返回：allSettled 返回包含 status 和 value/reason 的对象数组。
- 容错性：在并行请求多个 AI 模型对比结果时，通常用 allSettled 以免某个模型接口超时导致整体无结果。
- 性能：两者都是并行触发，不影响总耗时。

常见坑：

- 在 Promise.all 中没有 catch 单个请求，导致整个链路崩溃。

二面（深入原理与项目）

1. AI 聊天界面的长列表通常会遇到性能瓶颈，你如何实现一个支持流式输出的虚拟列表？

难度：★★★

考点：性能优化、虚拟列表、动态高度

答案要点：

流式输出会导致列表项高度动态变化，这给传统的虚拟列表带来了挑战。

- 动态高度计算：使用 `ResizeObserver` 监听每个消息组件的高度变化并更新偏移量缓存。
- 滚动锁定：当用户在底部时，新内容进入需自动滚动；当用户往上翻阅历史时，需锁定滚动位置。
- 渲染频率控制：流式数据更新极快，需通过 `requestAnimationFrame` 或节流减少 React/Vue 的渲染次数。
- 预估高度：初始给定一个预估值，待实际渲染后修正。

常见坑：

- 频繁更新 state 导致严重的掉帧。
- 滚动条抖动问题。

2. 在前端如何实现 Prompt 的版本管理和动态替换变量？

难度：★★

考点：字符串模板、应用逻辑设计

答案要点：

前端通常需要一套机制来管理复杂的 Prompt 模板，并根据用户输入填充变量。

- 模板引擎：采用类似 Mustache 的占位符语法 `{{variable}}`。
- 结构化管理：将 Prompt 存储为 JSON 配置，包含版本号、角色设定、示例（Few-shot）。
- 动态注入：编写工具函数，正则匹配占位符并从当前上下文中提取数据替换。
- 安全校验：防止用户输入中包含特殊指令字符破坏 Prompt 结构。

常见坑：

- 硬编码 Prompt 导致难以维护。
- 变量替换时没处理特殊字符转义。

3. 谈谈你对 Function Calling（函数调用）前端实现流程的理解。

难度：★★★

考点：AI Agent、交互链路

答案要点：

Function Calling 是 AI 模型决定调用外部工具的过程，前端需要处理中间状态和 UI 反馈。

- 响应识别：判断模型返回的 `finish_reason` 是否为 `tool_calls`。
- 逻辑执行：前端根据模型给出的函数名和参数，执行本地定义的 JS 函数（如查询天气、绘图）。

- 结果回传：将函数执行结果再次发送给模型，请求其生成最终的自然语言描述。
- UI 交互：在调用过程中展示“正在搜索/正在计算”的中间状态组件。

常见坑：

- 递归调用死循环。
- 函数执行失败时没有妥善告知模型。

4. 如何优化大模型首字响应时间（TTFT）的前端感知？

难度：★★★

考点：用户体验、性能指标

答案要点：

TTFT 是 AI 应用的关键指标，前端可以通过多种手段降低用户的焦虑感。

- 骨架屏与占位：在请求发出瞬间展示打字机光标或加载动画。
- 边缘计算：利用 Vercel Edge Functions 等在地理位置更近的节点处理请求。
- 预连接：对 API 域名进行 dns-prefetch 和 preconnect。
- 乐观更新：对于简单的指令，先展示预期的 UI 变化。

常见坑：

- 过度依赖 Loading 动画而忽略了流式输出的即时性。

5. 如果 AI 返回的内容包含复杂的数学公式或 Mermaid 图表，前端如何集成相关插件？

难度：★★

考点：第三方库集成、渲染引擎

答案要点：

需要扩展 Markdown 解析器的能力，通过插件机制支持特定语法。

- 公式渲染：集成 KaTeX 或 MathJax，识别 `$...$` 语法。
- 图表渲染：集成 mermaid.js，在 Markdown 解析阶段识别代码块语言为 mermaid 的部分。
- 异步加载：由于这些库体积较大，应采用动态 import 异步加载渲染引擎。
- 生命周期管理：在组件更新后手动触发渲染器的重新扫描。

常见坑：

- 重复渲染导致的性能开销。
- Mermaid 渲染失败导致整个页面白屏。

6. 你们是如何处理 AI 会话中的上下文长度限制（Context Window）的？

难度：★★★

考点：Token 管理、数据结构

答案要点：

前端需要参与上下文的剪裁策略，以保证请求不超出模型限制并节省 Token。

- Token 估算：使用 gpt-3-encoder 或 tiktoken 库在前端预估当前对话的 Token 数。
- 滑动窗口：当长度超限时，剔除最早的几轮对话，但保留 System Prompt。
- 摘要压缩：将历史对话发送给模型生成简短摘要，用摘要替代长历史。
- 优先级排序：保留最近的对话和最相关的上下文（如 RAG 检索到的内容）。

常见坑：

- 简单地按数组长度截取，导致截断了半个 JSON 或重要指令。

7. 实现一个带并发限制的 Promise 调度器，模拟同时调用多个 AI 代理。

难度：★★

考点：手写代码、异步并发

答案要点：

核心逻辑是维护一个执行队列，当正在运行的任务数小于限制时，从队列中取出新任务。

代码块

```
1  class Scheduler {
2    constructor(limit) {
3      this.limit = limit;
4      this.running = 0;
5      this.queue = [];
6    }
7    add(promiseCreator) {
8      return new Promise((resolve) => {
9        this.queue.push({ promiseCreator, resolve });
10       this.run();
11     });
12   }
13   run() {
14     while (this.running < this.limit && this.queue.length > 0) {
15       const { promiseCreator, resolve } = this.queue.shift();
16       this.running++;
17       promiseCreator().then(() => {
18         this.running--;
19         resolve();
20         this.run();
21       });
22     }
23   }
24 }
```

常见坑：

- 任务执行失败后没有减少 running 计数，导致调度器死锁。

8. 谈谈前端如何防范 Prompt 注入攻击？

难度：★★

考点：安全、AI 特有风险

答案要点：

Prompt 注入是指用户通过输入恶意指令误导模型跳出设定。

- 输入过滤：在前端对用户输入进行关键词扫描（如“忽略上述指令”）。
- 结构化封装：不要直接拼接字符串，而是使用 Chat Completion 接口的角色划分（System/User）。
- 输出校验：对 AI 返回的内容进行二次验证，确保其符合预期的格式（如 JSON）。
- 权限隔离：AI 执行的 Function Calling 必须经过用户二次确认。

三面（架构与综合能力）

1. 如果让你设计一个企业级的 AI Agent workflow编排系统（类似 LangFlow），你会如何设计前端架构？

难度：★★★★

考点：系统设计、复杂应用架构

答案要点：

这是一个典型的图形编辑类应用，核心在于状态管理和节点通信。

- 视图层：基于 React Flow 或 X6 实现节点拖拽、连线和画布缩放。
- 数据模型：采用图结构（Nodes & Edges）存储，使用 Zustand 或 Redux 管理全局状态。
- 协议层：定义一套标准化的 JSON Schema，描述每个节点的输入、输出和配置参数。
- 插件化：每个 AI 能力（LLM、Search、Python Code）封装为独立组件，通过动态加载实现扩展。
- 预览机制：实现一个侧边栏沙箱，实时模拟当前工作流的执行过程。

常见坑：

- 节点过多时的 Canvas 渲染性能问题。
- 循环引用的检测逻辑缺失。

2. 在 AI 交互中，如何定义一套衡量“前端交互质量”的指标体系？

难度：★★★★

考点：方法论、数据驱动

答案要点：

除了传统的性能指标，AI 场景需要关注交互的连贯性和准确性。

- 响应延迟：TTFT（首字时间）、整个回复完成时间。
- 交互转化：点赞/点踩率、重新生成率、复制率。
- 错误率：流式中断率、解析失败率、函数调用失败率。
- 体验指标：打字机动画的平滑度、光标跳动频率。

追问：

- 如果发现某个版本的 Prompt 导致用户点踩率上升，你会如何排查？

3. 如何解决 AI 场景下前端资源体积过大（如集成 WebLLM 或大型渲染库）的问题？

难度：★★

考点：工程化、性能优化

答案要点：

AI 应用常涉及重型资源，需从加载策略和缓存入手。

- 分包策略：利用 Vite/Webpack 的 SplitChunks 将 AI 相关的库独立打包。
- 离线存储：利用 Service Worker 和 Cache API 缓存模型权重文件。
- 按需加载：仅在用户切换到特定功能（如绘图）时才加载对应的渲染引擎。
- 预加载：在用户输入时，利用 idle 状态预加载可能用到的资源。

4. 谈谈 AI 时代下，前端工程师的角色发生了哪些转变？你如何看待“AI 会取代前端”这个观点？

难度：★★

考点：行业视野、职业思考

答案要点：

前端正从“UI 开发”向“交互编排”转变，AI 是工具而非替代者。

- 职责变化：从关注像素完美到关注 Prompt 调试、数据流编排、模型能力边界。
- 效率提升：利用 Copilot 等工具处理重复性代码，将精力转向系统设计。
- 核心价值：复杂业务逻辑的抽象、极致的用户体验、多端适配的复杂性，这些仍需人类工程师。
- 结论：AI 提高了门槛，淘汰的是低端重复性劳动，但对高级架构能力的需求反而增加。

5. 聊聊你在过去一年中遇到的最难的 AI 相关前端技术挑战，你是如何解决的？

难度：★★★

考点：项目经验、解决问题能力

答案要点：

（此题无标准答案，重点考察逻辑）

- 背景：描述一个具体的 AI 场景（如：流式 Markdown 渲染卡顿、Agent 状态机过于复杂等）。
- 方案：对比了哪几种方案（如：对比了虚拟列表和普通列表）。
- 结果：最终通过什么手段（如：引入了 Web Worker 处理解析逻辑）提升了多少性能指标。
- 总结：沉淀了什么通用的组件或工具。

6. 如果大模型支持 1M 的上下文，前端在数据展示和交互上需要做哪些适配？

难度：★★★

考点：前瞻性思考、大数据量处理

答案要点：

超长上下文意味着前端不能再简单地一次性渲染所有内容。

- 导航增强：引入 AI 自动生成的目录或时间轴，方便用户快速定位。
- 局部渲染：采用类似 IDE 的分段加载机制，仅渲染可视区域附近的对话。
- 搜索与过滤：提供强大的全文搜索和基于语义的上下文过滤。
- 内存管理：及时销毁不可见区域的复杂组件实例，防止浏览器内存溢出。